Data Structure and Algorithms (CS221)

Week 01

Said Nabi

said.nabi@giki.edu.pk



Faculty of Computer Science and engineering

Gik Institute Topi, Swabi

Introduction

Engr. Said Nabi

Introduction: Course Instructor

Education:

BSSE: UoM

MSSE: Comsats University Isamabad

Speciallization: e-Commerce Development,

MS Thesis: e-Commerce Product Recommendadtion Using Machine Learning Algorithms

Practicle Experience:

Working as Freelancer since 2016

Become TRS in 2020 on Fiverr

Completed over 800+ Projects with 500+ 5 stars reviews

Created a store for my Client, made $100k sales a month

Instructor

Email: said.nabi@giki.edu.pk

Webpage: https://tecplace.net/

Office: S20

Telephone Extension: 2154

Office Hours:

Monday/Thursday	9:00 AM - 4:00 PM

Course TA

- Still to be assigned
- Never hesitate to contact TA or me whenever you have some problem
- The **worst question** is the one that stays in your head **unanswered**.

Some Rules

- There is only one rule that will make the rest of the rules
  - Raise your hand before asking any question and then WAIT for the permission
  - **Never ever Miss a class**
  - **Never ever “sleep” in the class**
  - Always communicate in the official communication language
  - Never even think about using mobile during the class

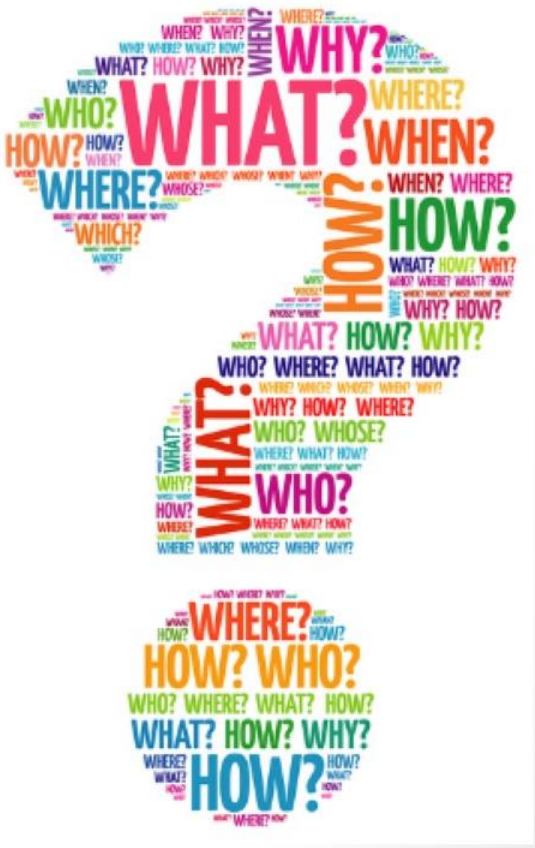
Class Rules [1/2]



- Be Punctual
  - Late comers are not allowed
  - 80% attendance is compulsory
- Be Attentive
- Be Prompt
  - Ready to learn
  - Class participation
  - Surprised quizzes
- Be Polite
  - Soft-spoken
  - No laughing

Class Rules [2/2]

- Be Honest
  - With yourself
  - Credit others
  - No cheating
  - No wastage of time
- Be Responsible
  - SWITCH OFF your phone
  - Penalty: treat for the whole class
- **Ask the questions as many as you can!!**



General Rule



- Course Instructor: Facilitator role
- The required characteristics from students: Dedication, passion, and continuous effort
- Focus: What else?

Dishonesty, Plagiarism in Quizzes, Assignments



- All individuals involved in any kind of cheating in any exam, quiz, assignment or project will get **-50%** score.

Academic Dishonesty (GIKI-SC-20)

Academic dishonesty, such as cheating, plagiarism, fabrication, and fraud, is prohibited. See Part III for specific definitions of academic dishonesty.

****

Attendance

- Students are required to attend 100% classes of courses registered and **cannot abstain from even a single class**.

- For circumstances beyond their control, students must apply for leave on prescribed Leave Application form.

Quizzes

- Announced
- Unannounced

Tentative Evaluation Breakdown

Assignments (4)	10
Quizzes (4)	10
Term Project	10
Midterm (1)	30
Final (1)	40
Total	100

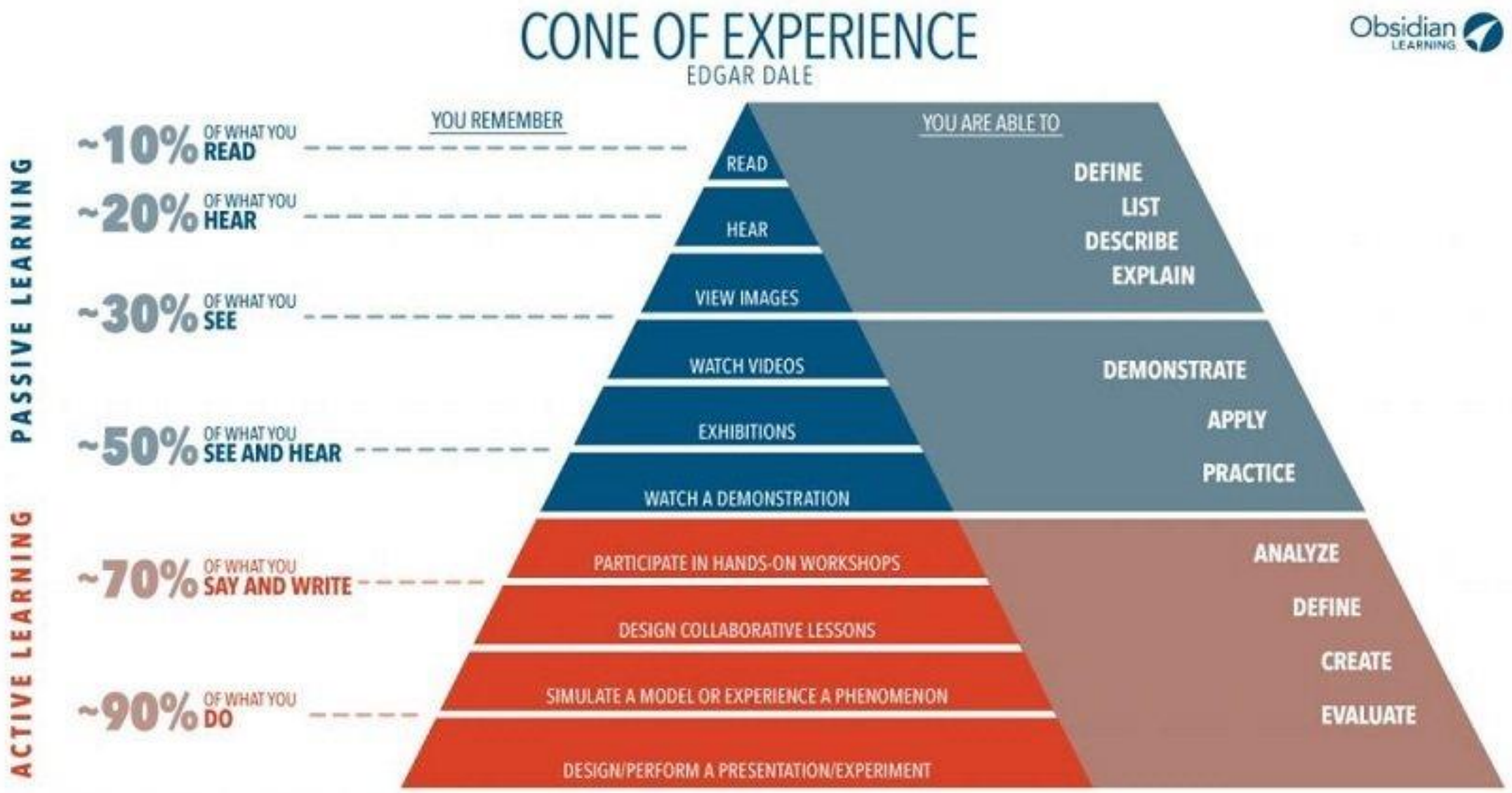
Course Execution

- Three lectures of one hour every week
- 1 Lab of 3 hours
- Course website: http://cms.giki.edu.pk

See schedule

Mind it: Lecture scheduling is to be done by the Dir. (A&E). Please avoid visiting me for any clashes

Remember


CONE OF EXPERIENCE
EDGAR DALE
Obsidian LEARNING
YOU REMEMBER
YOU ARE ABLE TO
PASSIVE LEARNING
ACTIVE LEARNING
~10% OF WHAT YOU READ
~20% OF WHAT YOU HEAR
~30% OF WHAT YOU SEE
~50% OF WHAT YOU SEE AND HEAR
~70% OF WHAT YOU SAY AND WRITE
~90% OF WHAT YOU DO
READ
HEAR
VIEW IMAGES
WATCH VIDEOS
EXHIBITIONS
WATCH A DEMONSTRATION
PARTICIPATE IN HANDS-ON WORKSHOPS
DESIGN COLLABORATIVE LESSONS
SIMULATE A MODEL OR EXPERIENCE A PHENOMENON
DESIGN/PERFORM A PRESENTATION/EXPERIMENT
DEFINE
LIST
DESCRIBE
EXPLAIN
DEMONSTRATE
APPLY
PRACTICE
ANALYZE
DEFINE
CREATE
EVALUATE

Outline

- Introduction to the Course
- Course Objectives
- Why this course is important for you?
- Course Contents and Structure
- Course Project

Lecture Format

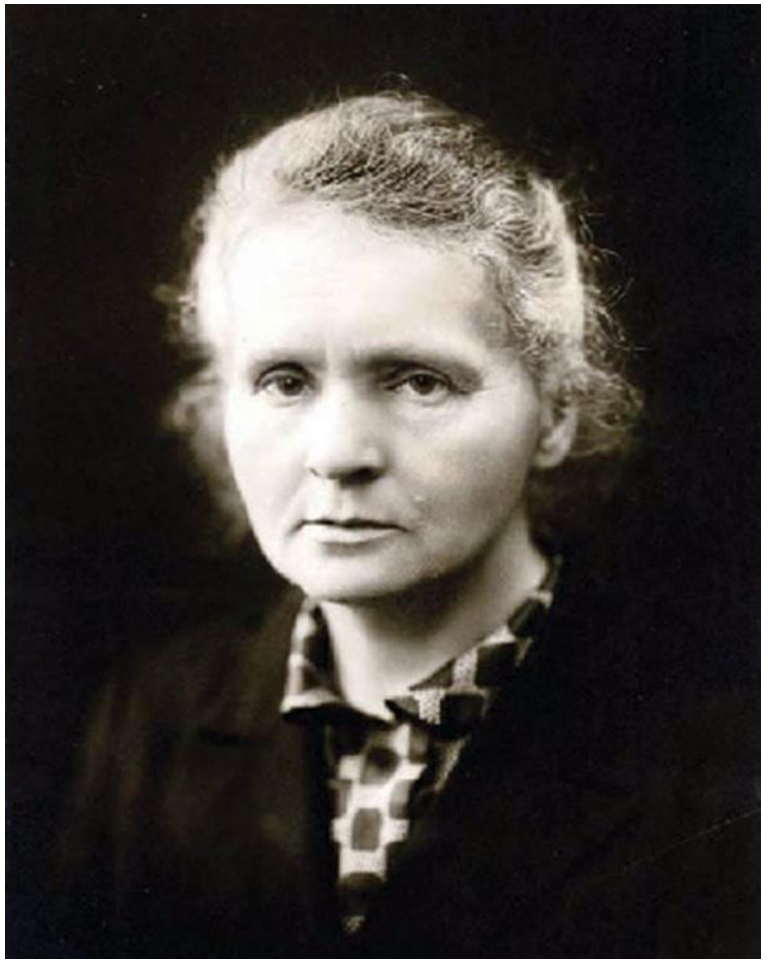
- Start with Q & A
- Main Lecture
  - No short breaks in the middle
- Practice sessions on various problems

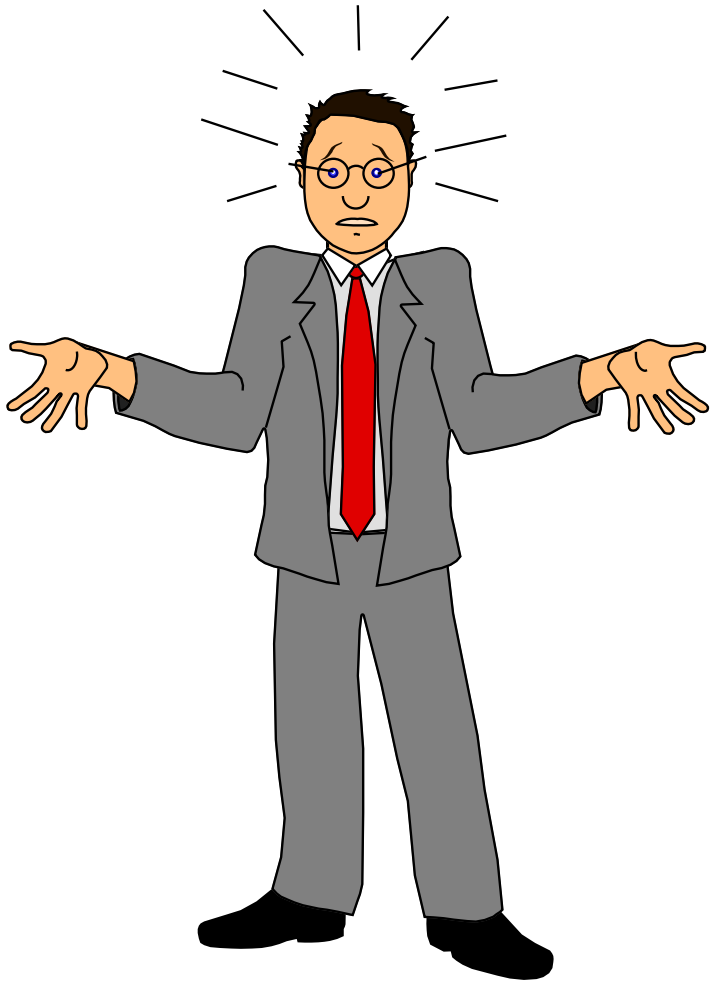
Course Outline

- In this course you will learn
  - Introduction
  - Algorithm Analysis
  - Abstract Data Types (ADTs)
  - Lists, Stacks, and Queues
  - Trees
  - Hashing
  - Queues
  - Sorting
  - Graph Algorithms
  - Algorithm Design Techniques

Motivation!

- **Nothing** in life is to be feared, it is only to be understood.” – Marie Curie
- DS & Algorithms = backbone of computing
- Without them, even fast hardware is useless

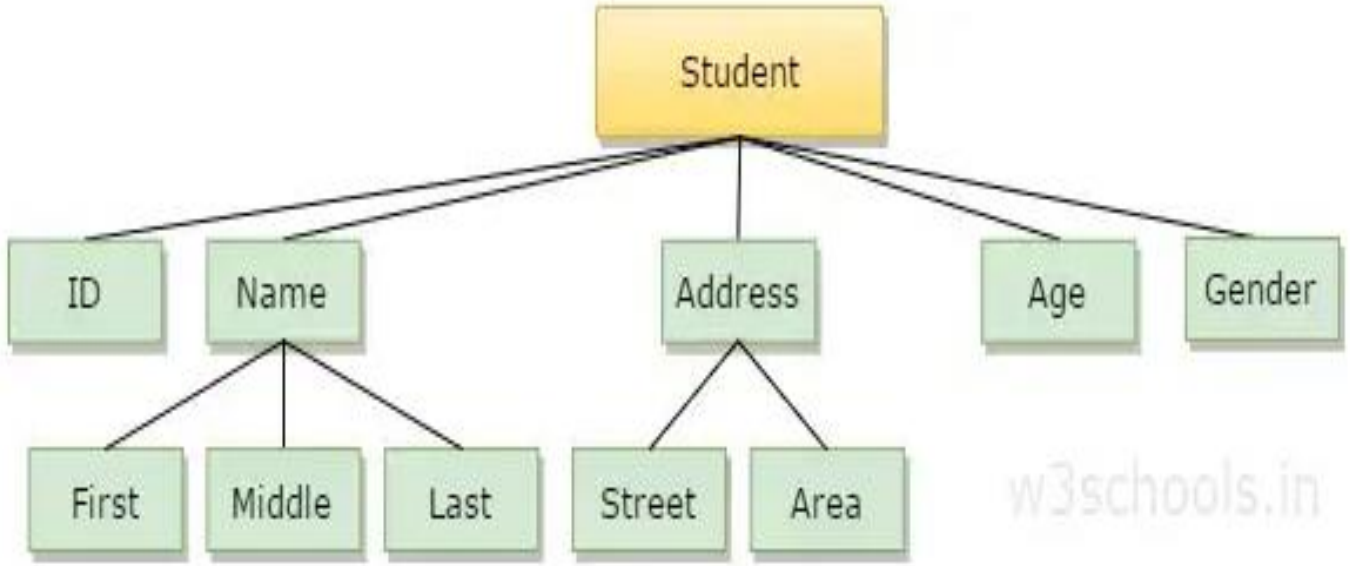




What exactly data structures and algorithms are?

What is data?

- Raw facts & figures
- Represented in numbers, text, images
- Organized = information



Real World Example

Let say you want to prepare Bana Milkshake for the first Time

You require a list of ingredients and a set of instructions presented in a comprehensible language that you can understand.

Ingredients: Data Structure

2 ripe bananas
1 cup of milk
1-2 tablespoons of sugar
Ice cubes

Instructions: Algorithm

- Peel and chop the bananas.
- Blend bananas, milk, sugar, and ice cubes until smooth.
- Adjust sugar to taste.
- Pour into glasses and enjoy!

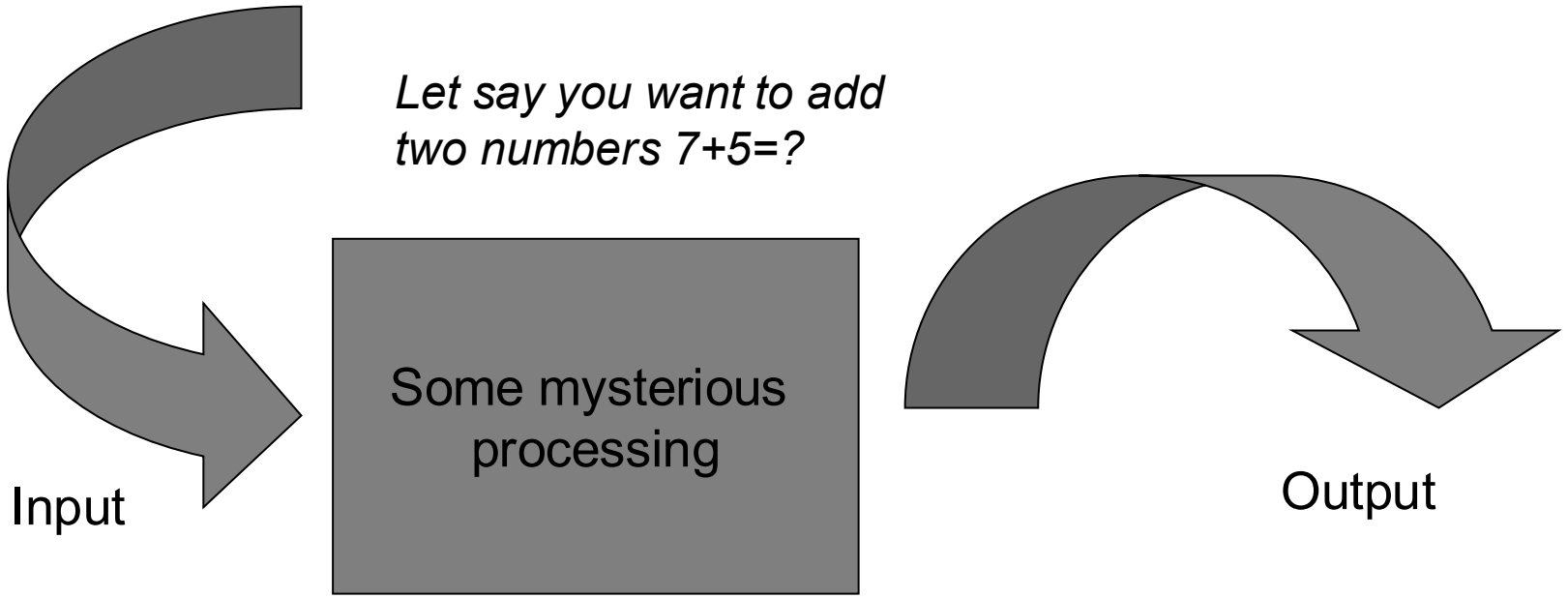


ingredients + recipe = dish → data structure + algorithm = program.

Similarly in computer World: You need to provide instructions which computer can understand.

To perform a task:

1. *Data Structure*
2. *Algorithm*



What is a data structure?

- In a general sense, any representation that is used for storing information is a data structure

- Example: An integer, structures, classes, linked lists, etc

- More typically, a *data structure* provides a way of *organization for a collection of data items*

Data Structure and algorithms

- **Data Sturcture**: is the way to arange data in main memory (ram)

  for efficient usage. (ingrediants)

- **Algorithm**: is a sequance of steps to solve a given problem. As seen in afore mentioned example of preparing banana milkshake.

Examples: Arrays, Linklists Stack, Queue,Tree, Graphs

Where Data Structure is Helpful?

- The choice of efficient data structure makes the difference between a program running in a few seconds or many days

Computer Based Example:

How to solve the following problems:

1. Input 3 numbers, print out the maximum.
2. Input 30000 numbers, print out the largest 10 numbers.

What is Data Structure Efficiency?

- A solution is said to be efficient if it solves the problem within its resource constraints in a reasonable amount of time.
  - Space
  - Time

- The cost of a solution is the amount of resources that the solution consumes.

Costs and Benefits

- Each data structure has costs and benefits.
- It is very difficult to find a data structure that is better than others in all situations.
- A data structure requires:
  - space for each data item it stores,
  - time to perform each basic operation,
  - programming effort.

Goals of this Course

1. Learn the commonly used data structures.
   - These form a programmer's basic data structure “toolkit”

2. Case Studies of Data Structures.

3. We will examine the costs and benefits of every data structure or program.

Example

- A cellular service company provides contracts to its 10 million users
- Due to new security enforcements, the company wants to **prevent issuing of multiple contracts to users**

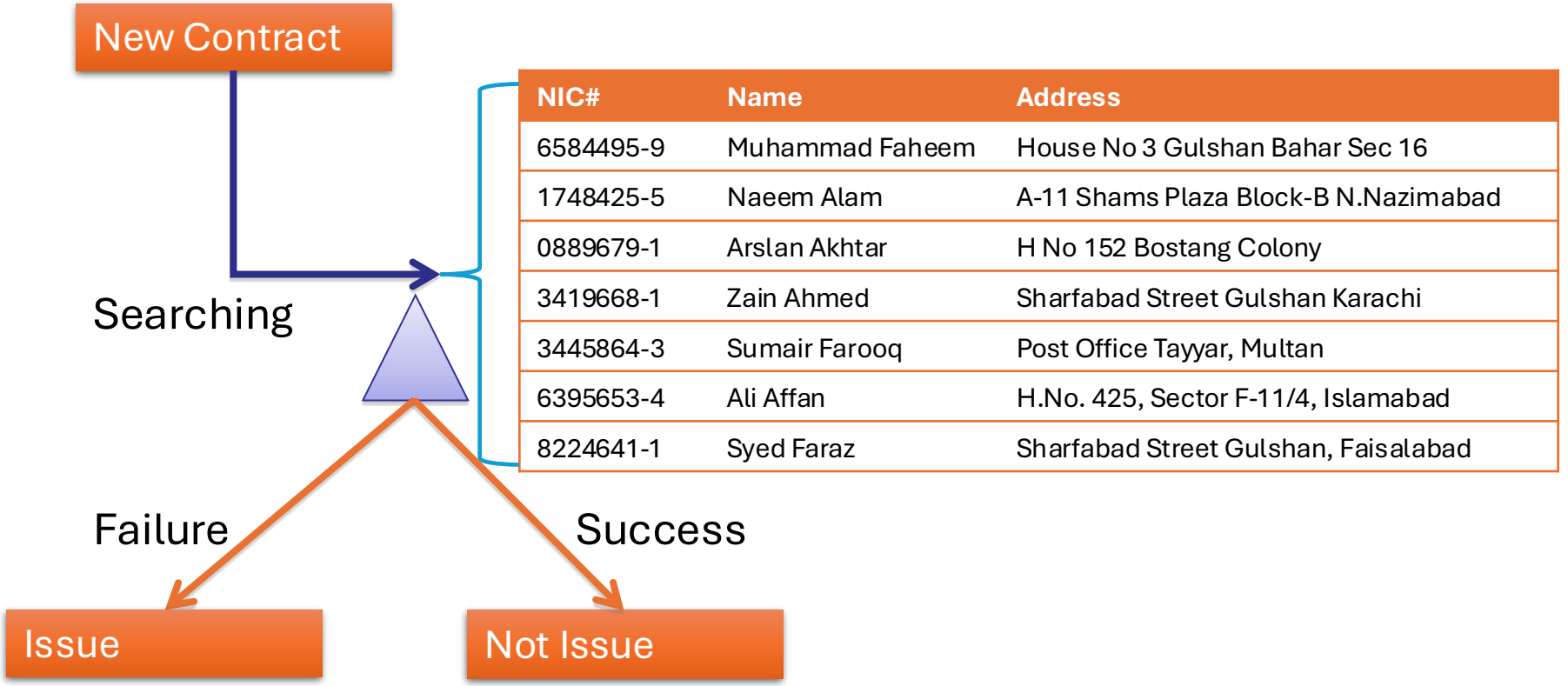
- Method of Detecting Multiple Contracts
  - Before issuing a new contract to user
  - First search the ***id of user*** in existing contracts database
  - In case of failure, issue a new contract
  - In case of success, do not issue a new contract to user

Example

NIC#	Name	Address
6584495-9	Muhammad Faheem	House No 3 Gulshan Bahar Sec 16
1748425-5	Naeem Alam	A-11 Shams Plaza Block-B N.Nazimabad
0889679-1	Arslan Akhtar	H No 152 Bostang Colony
3419668-1	Zain Ahmed	Sharfabad Street Gulshan Karachi
3445864-3	Sumair Farooq	Post Office Tayyar, Multan
6395653-4	Ali Affan	H.No. 425, Sector F-11/4, Islamabad
8224641-1	Syed Faraz	Sharfabad Street Gulshan, Faisalabad

- Linear Array (with 10 million entries)
  - 3 arrays (NIC, name, address),
  - structure array,
  - class's object array

Example



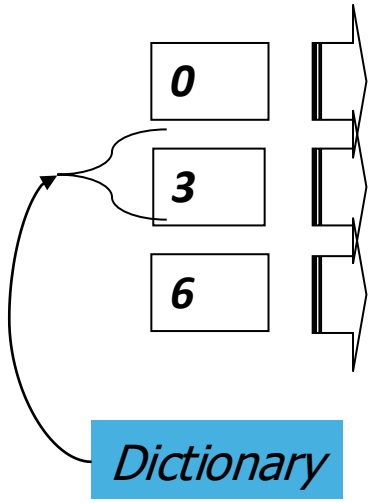
NIC#	Name	Address
6584495-9	Muhammad Faheem	House No 3 Gulshan Bahar Sec 16
1748425-5	Naeem Alam	A-11 Shams Plaza Block-B N.Nazimabad
0889679-1	Arslan Akhtar	H No 152 Bostang Colony
3419668-1	Zain Ahmed	Sharfabad Street Gulshan Karachi
3445864-3	Sumair Farooq	Post Office Tayyar, Multan
6395653-4	Ali Affan	H.No. 425, Sector F-11/4, Islamabad
8224641-1	Syed Faraz	Sharfabad Street Gulshan, Faisalabad

- Any disadvantage of Linear Array (Data Structure)?
- How to improve?

Example

- Improved Data Structure
  - Create a dictionary data structure
  - Group all those records together that start with similar NIC (first digit) numbers, and add a dictionary entry for each distinct digit (0-9)
  - Example: 3419668-1, 3445864-3, 1748425-5.
    - 3 and 1 are dictionary entries
  - In case of searching, first search the dictionary entry, and then proceed to searching contracts

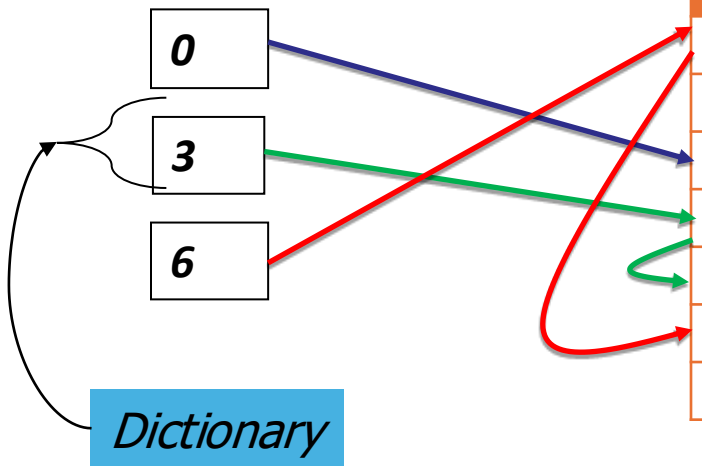
Example



NIC#	Name	Address
6584495-9	Muhammad Faheem	House No 3 Gulshan Bahar Sec 16
1748425-5	Naeem Alam	A-11 Shams Plaza Block-B N.Nazimabad
0889679-1	Arslan Akhtar	H No 152 Bostang Colony
3419668-1	Zain Ahmed	Sharfabad Street Gulshan Karachi
3445864-3	Sumair Farooq	Post Office Tayyar, Multan
6395653-4	Ali Affan	H.No. 425, Sector F-11/4, Islamabad
8224641-1	Syed Faraz	Sharfabad Street Gulshan, Faisalabad

Example

- Another Data Structure
  - Maintain pointers with records
  - Non NULL pointer indicates presence of next record



NIC#	Name	Address
6584495-9	Muhammad Faheem	House No 3 Gulshan Bahar Sec 16
1748425-5	Naeem Alam	A-11 Shams Plaza Block-B N.Nazimabad
0889679-1	Arslan Akhtar	H No 152 Bostang Colony
3419668-1	Zain Ahmed	Sharfabad Street Gulshan Karachi
3445864-3	Sumair Farooq	Post Office Tayyar, Multan
6395653-4	Ali Affan	H.No. 425, Sector F-11/4, Islamabad
8224641-1	Syed Faraz	Sharfabad Street Gulshan, Faisalabad

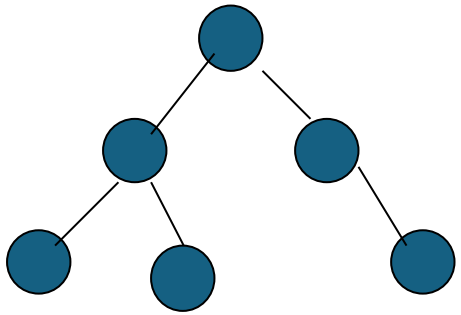
Data structures let the input and output be represented in a way that can be handled efficiently and effectively.



array



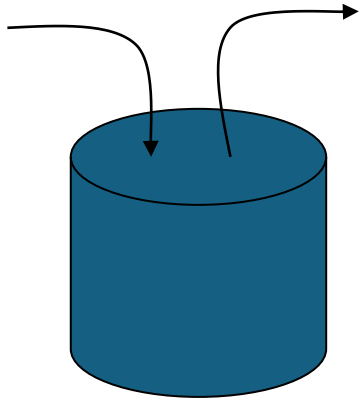
Linked list



tree



queue



stack

Definition

- Data may be organized in different ways
- Data structure is the logical or mathematical model of a particular organization of data
    - Must be rich enough to mirror the actual relationships of the data in the real-world sample

Arrays

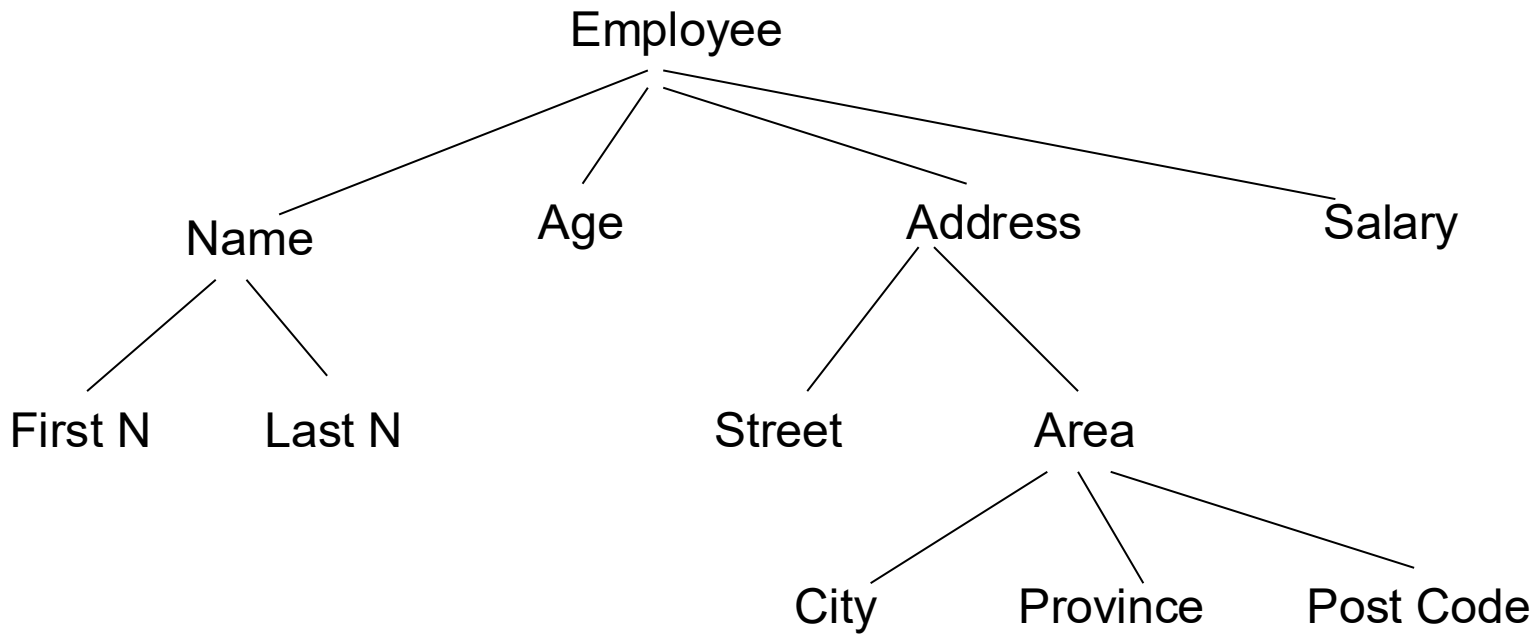


Liner Arrays

Link List

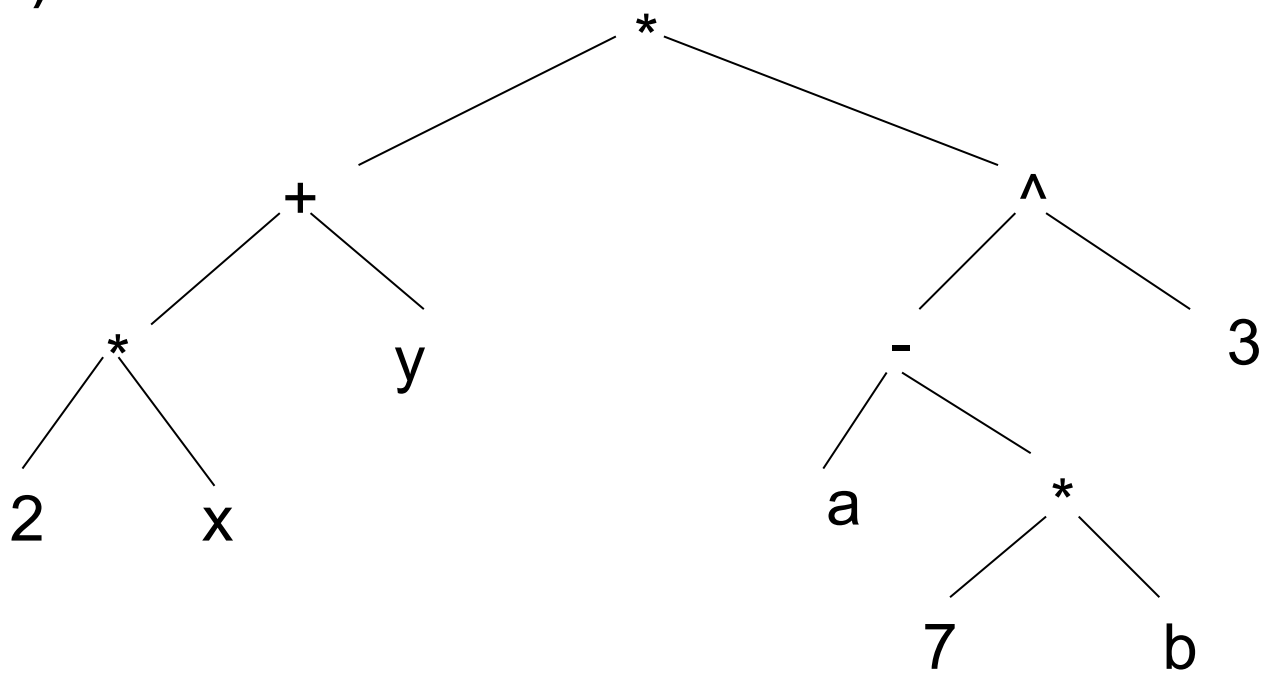
Jamal	1			
Sana	1			
		Saeed	2	
Farooq	3			
Salman	3		Salman	3

Trees

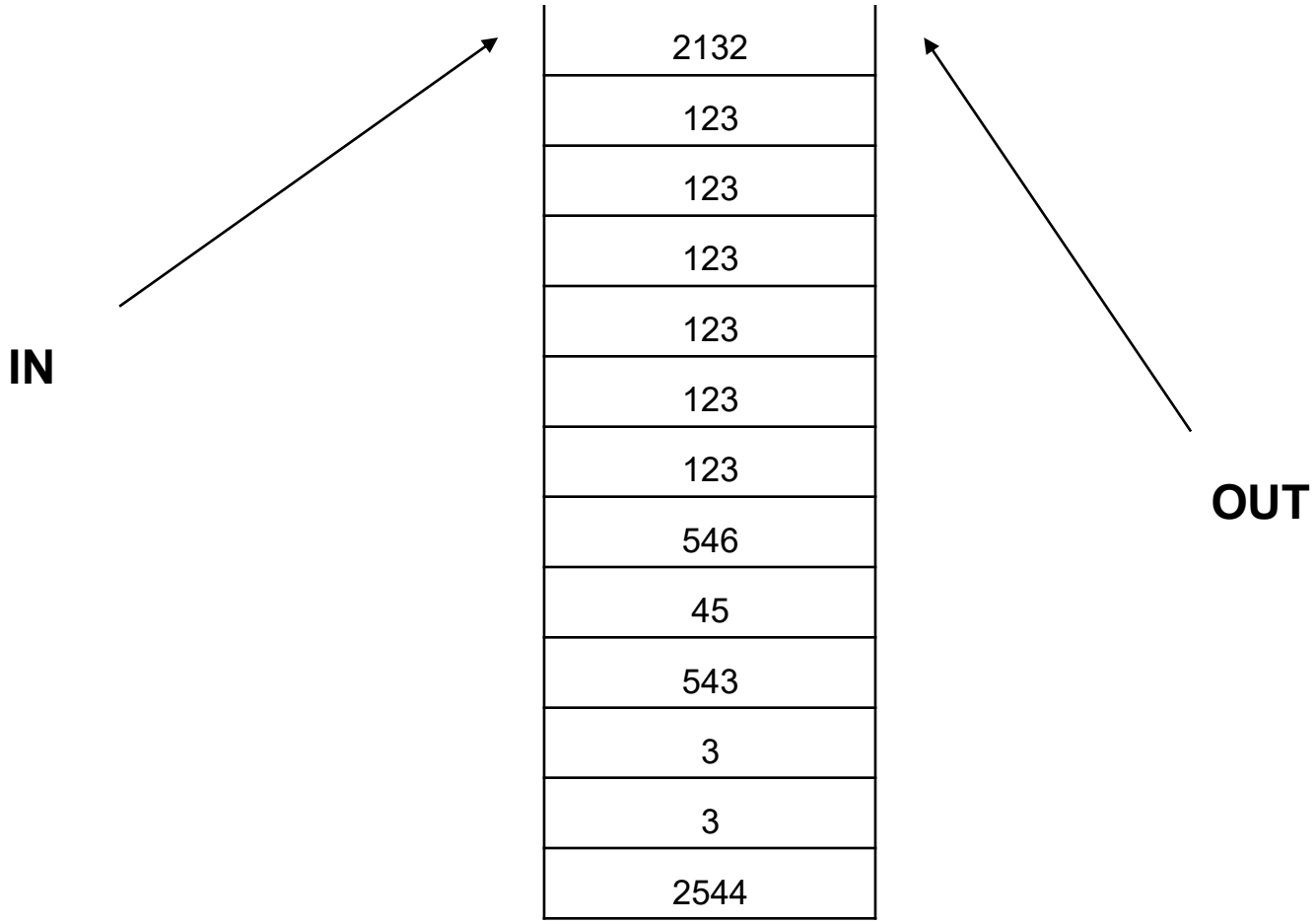

Employee
Name
Age
Address
Salary
First N
Last N
Street
Area
City
Province
Post Code

Trees

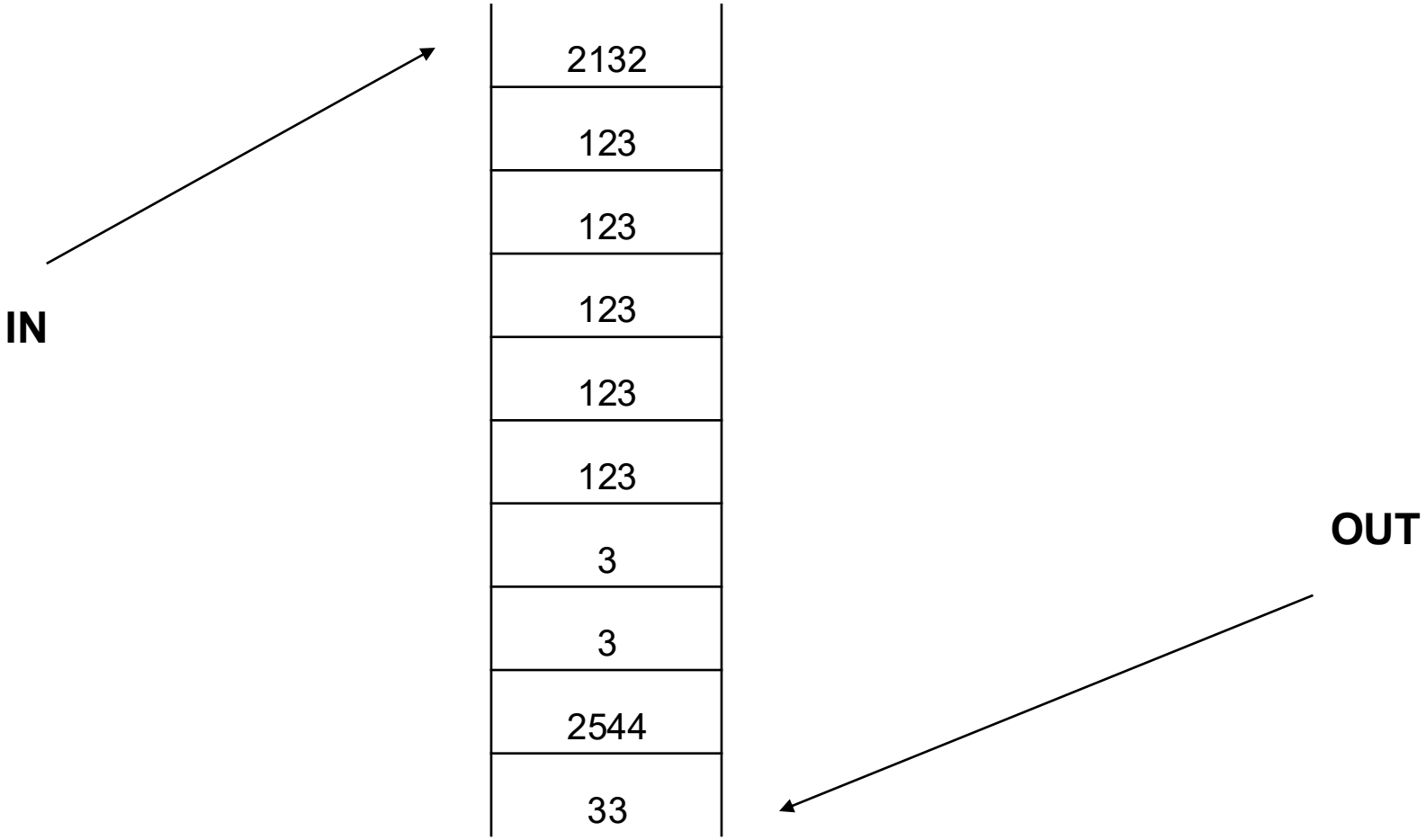
$(2x+y)(a-7b)^3$



Stacks



Queue



The Need for Data Structures

- Goal: to **organize data**

- Criteria: to facilitate **efficient**
  - **storage** of data
  - **retrieval** of data
  - **manipulation** of data

- Design Issue:
  - **select and design** appropriate data types.
    (This is the real essence of OOP.)

Data Structure Operations

- Insertion
- Deletion
- Update
- Searching
- Sorting
- Merging

Abstract Data Types

- Abstract Data Type (ADT): a definition for a **data type** solely in terms of a **set of values** and a **set of operations** on that data type.

- Each ADT **operation** is defined by its inputs and outputs.
  - Hide Implementation details

Abstract Data Types

Def.

a collection of related data items
together with
an associated set of operations

e.g. whole numbers (integers)
Basic Operations: apply arithmetic operations (addition, subtraction, multiplication and division)

e.g. Flight reservation (`seats`)
Basic Operations: find empty seat, reserve a seat,
cancel a seat assignment

Why "abstract" ?
Data, operations, and relations are studied **independent of implementation.**

What not ***how*** is the focus.

Data Structure

- A data structure is the physical implementation of an ADT.
  - Each operation associated with the ADT is implemented by one or more subroutines in the implementation.

- Data structure usually refers to an organization of data in main memory.

- File structure is an organization for data on peripheral storage, such as a disk drive.

ADT's for Primitive Data Types

Boolean data

Data values: {`false`, `true`}

Operations:

and	`&&`
or	`||`
not	!

&&

0	**0**	**0**
1	**1**	**1**
1	**0**	**0**

||

0	**0**	**0**
0	**1**	**1**
1	**1**	**1**

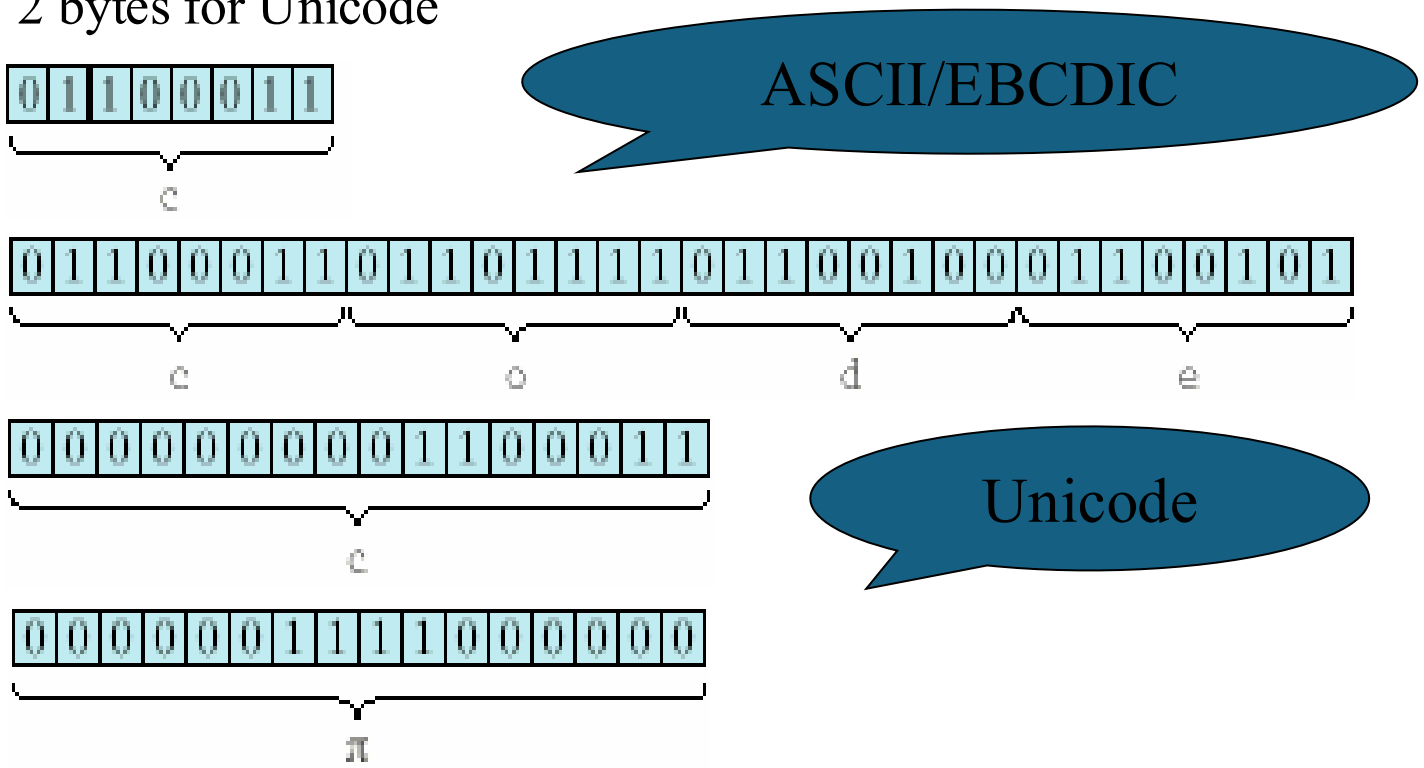
x	$!x$
0	1
1	0

Character Data

Store numeric codes (ASCII, EBCDIC, Unicode)

1 byte for ASCII and EBCDIC,

2 bytes for Unicode



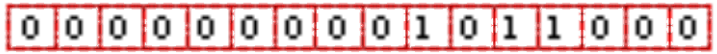
Basic operation: comparison to determine ifEqual, Less than ,Greater than, etc. use their numeric codes (i.e. use ordinal value)

Integer Data

Non-negative (unsigned) integer:

Store its base-two representation in a fixed number w of bits
(e.g., w = 16 or w = 32)

88 = 0000000001011000_2



Signed integer:
Store in a fixed number w of bits using complement representations:

Operations: arithmetic operators for addition, subtraction, multiplication and division

Time Complexity

Algorithm

- Finite sequence of instructions.

An input should not take the program in an infinite loop

- Each instruction having a clear meaning.

Very subjective. What is clear to me, may not be clear to you.

- Each instruction requiring finite amount of effort.

Very subjective. Finite on a supercomputer or microcomputer?

- Each instruction requiring finite time to complete.

Very subjective. 1 min, 1 hr, 1 year or a lifetime?

Algorithm Definition

A finite set of statements that guarantees an optimal solution in finite interval of time

Good Algorithms?

- Run in less time
- Consume less memory

But computational resources (time complexity) is usually more important

Measuring Efficiency

- The efficiency of an algorithm is a measure of the amount of resources consumed in solving a problem of size n.
    - The resource we are most interested in is time
    - We can use the same techniques to analyze the consumption of other resources, such as memory space.
- It would seem that the most obvious way to measure the efficiency of an algorithm is to run it and measure how much processor time is needed
- Is it correct

Factors

- Hardware
- Operating System
- Compiler
- Size of input
- Nature of Input
- Algorithm

  Which one should be improved?

Running Time of an Algorithm

- Depends upon
  - Input Size
  - Nature of Input
- Generally time grows with size of input, so running time of an algorithm is usually measured as function of input size.
- Running time is measured in terms of number of steps/primitive operations performed
- Independent from machine, OS

Finding running time of an Algorithm / Analyzing an Algorithm

- Running time is measured by number of steps/primitive operations performed
- Steps means elementary operation like
  - ,+, *,<, =, A[i] etc
- We will measure number of steps taken in term of size of input

Simple Example (1)

```
// Input: int A[N], array of N integers
// Output: Sum of all numbers in array A

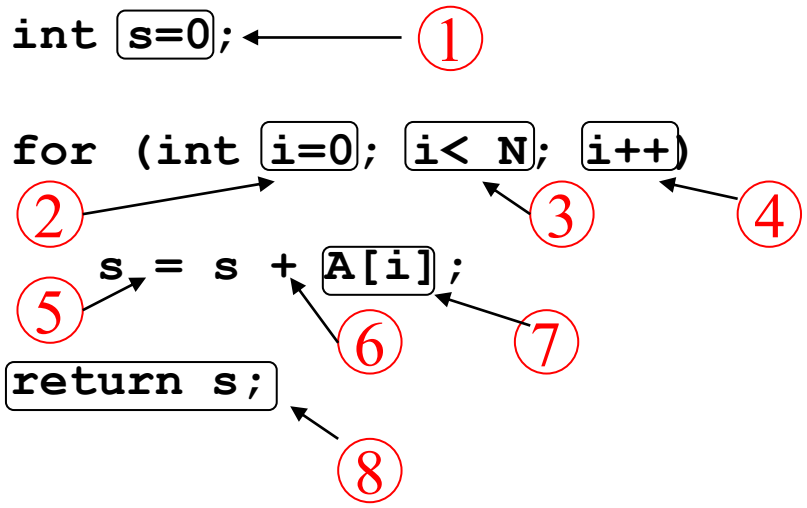
int Sum(int A[], int N)
{
1.   int s=0;
2.   for (int i=0; i< N; i++)
3.      s = s + A[i];
4.   return s;
}
```

How should we analyse this?

Simple Example (2)

```
// Input: int A[N], array of N integers
// Output: Sum of all numbers in array A

int Sum(int A[], int N){
    int s=0;
    for (int i=0; i< N; i++)
        s = s + A[i];
    return s;
}
```



1,2,8: Once
3,4,5,6,7: Once per each iteration of for loop, N iteration
Total: 5N + 3
The *complexity function* of the algorithm is : $f(N) = 5N + 3$

Simple Example (3)
Growth of 5n+3

Estimated running time for different values of N:

N = 10 => 53 steps

N = 100 => 503 steps

N = 1,000 => 5003 steps

N = 1,000,000 => 5,000,003 steps

As N grows, the number of steps grow in *linear* proportion to N for this function *"Sum"*

What Dominates in Previous Example?

What about the +3 and 5 in 5N+3?

- As N gets large, the +3 becomes insignificant
- 5 is inaccurate, as different operations require varying amounts of time and also does not have any significant importance

What is fundamental is that the time is *linear* in N.
Asymptotic Complexity: As N gets large, concentrate on the highest order term:

- Drop lower order terms such as +3
- Drop the constant coefficient of the highest order term i.e. N

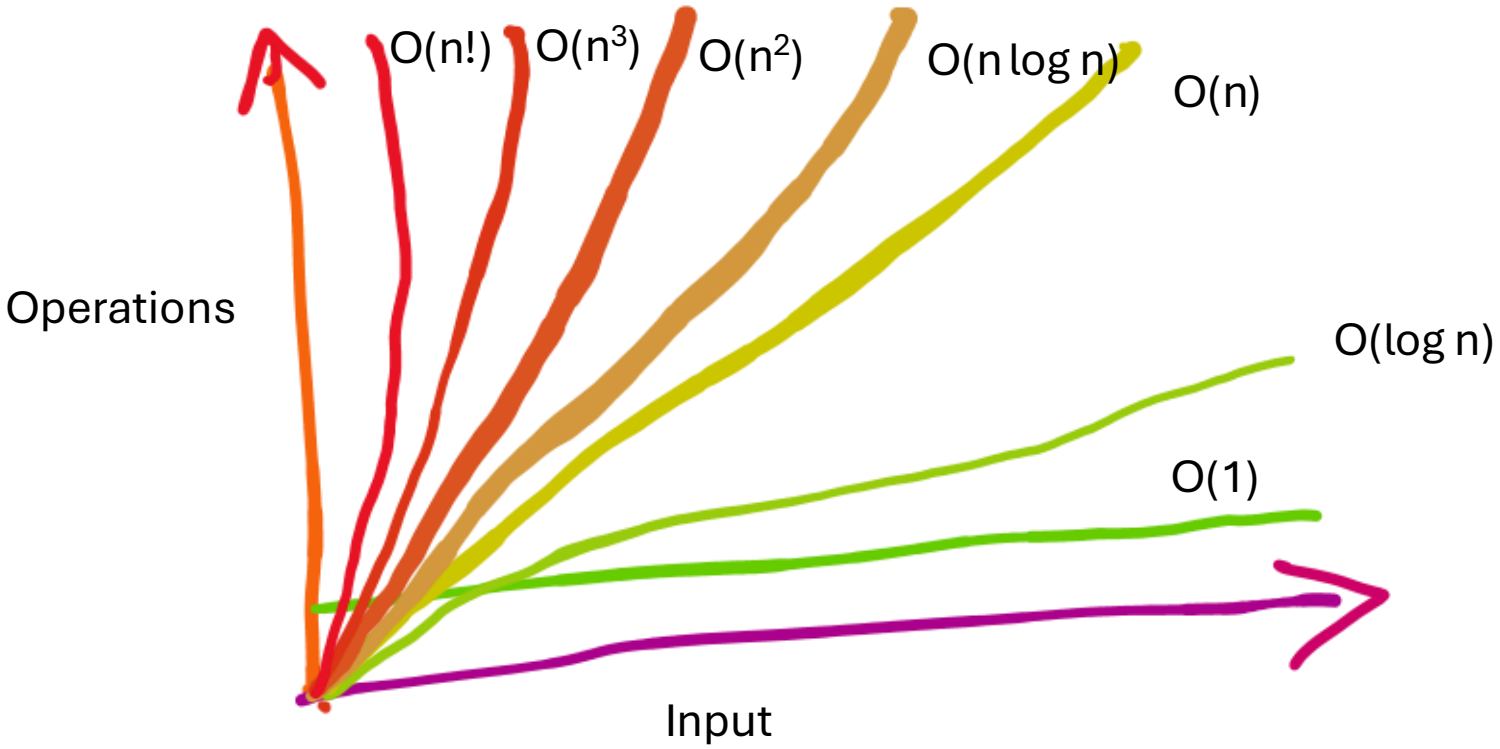
Be careful to differentiate between

Performance: how much time/memory/disk/… is actually used when a program is run. This depends on the machine, compiler, etc. as well as the code.

Complexity: how do the resource requirements of a program or algorithm scale, i.e., what happens as the size of the problem being solved gets larger.

Asymptotic Complexity

- The 5N+3 time bound is said to "grow asymptotically" like N
- This gives us an approximation of the complexity of the algorithm
- Ignores lots of (machine dependent) details, concentrate on the bigger picture



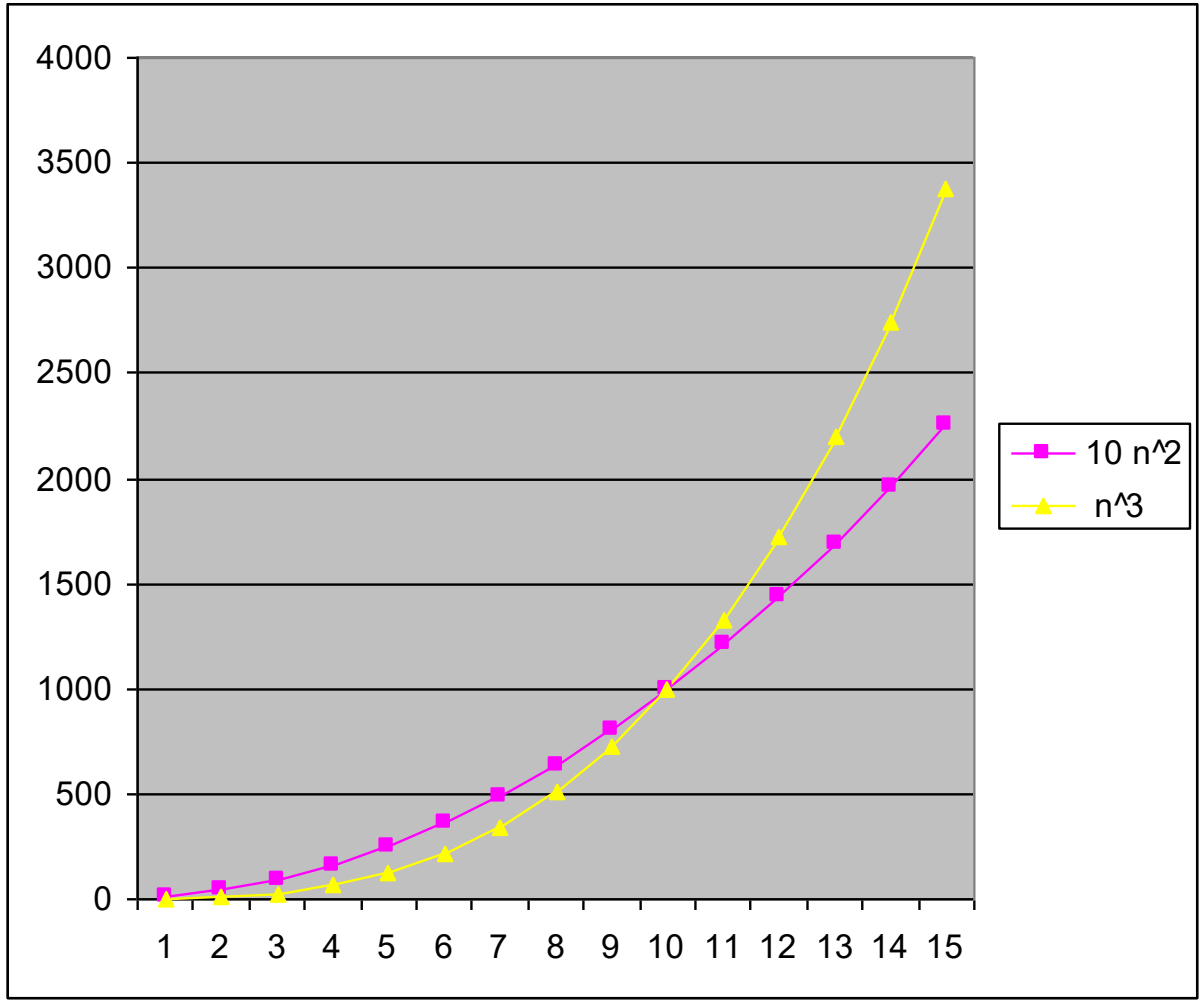
Comparing Functions: Asymptotic Notation

- Big Oh Notation: Upper bound
- Omega Notation: Lower bound
- Theta Notation: Tighter bound

Example : Comparing Functions

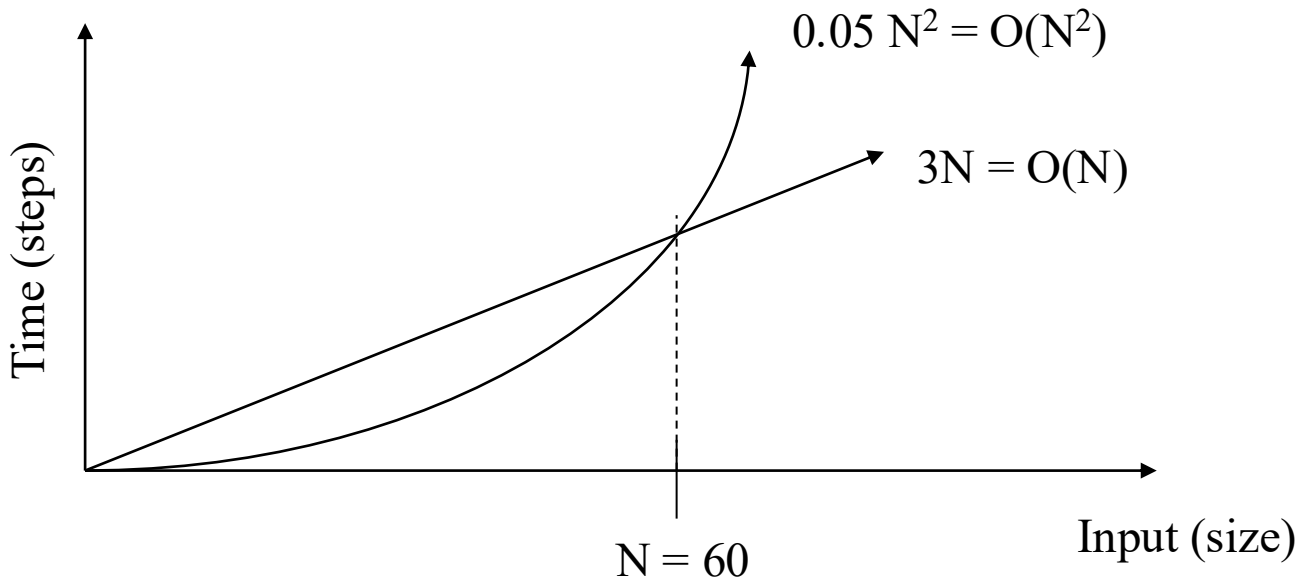
- Which function is better?

  $10\ n^2$ Vs n^3



Comparing Functions

- As inputs get larger, any algorithm of a smaller order will be more efficient than an algorithm of a larger order



Big-Oh Notation

- Even though it is correct to say "$7n - 3$ is $O(n^3)$", a better statement is "$7n - 3$ is $O(n)$", that is, one should make the approximation as tight as possible
- Simple Rule:

  Drop lower order terms and constant factors

  $7n-3$ is $O(n)$

  $8n^2 \log n + 5n^2 + n$ is $O(n^2 \log n)$

Performance Classification

f(*n*)	Classification
1	*Constant*: run time is fixed, and does not depend upon n. Most instructions are executed once, or only a few times, regardless of the amount of information being processed
log n	*Logarithmic*: when *n* increases, so does run time, but much slower. Common in programs which solve large problems by transforming them into smaller problems.
n	*Linear*: run time varies directly with *n*. Typically, a small amount of processing is done on each element.
n log n	When *n* doubles, run time slightly more than doubles. Common in programs which break a problem down into smaller sub-problems, solves them independently, then combines solutions
n^2	*Quadratic*: when *n* doubles, runtime increases fourfold. Practical only for small problems; typically the program processes all pairs of input (e.g. in a double nested loop).
n^3	*Cubic*: when n doubles, runtime increases eightfold
2^n	*Exponential*: when n doubles, run time squares. This is often the result of a natural, "brute force" solution.

Input size does matter

What happens if we double the input size N?

N	$\log_2 N$	5N	$N \log_2 N$	N^2	2^N
8	3	40	24	64	256
16	4	80	64	256	65536
32	5	160	160	1024	$\sim 10^9$
64	6	320	384	4096	$\sim 10^{19}$
128	7	640	896	16384	$\sim 10^{38}$
256	8	1280	2048	65536	$\sim 10^{76}$

Input size does matter

- Suppose a program has run time O(n!) and the run time for
  n = 10 is 1 second

  For n = 12, the run time is 2 minutes
  For n = 14, the run time is 6 hours
  For n = 16, the run time is 2 months
  For n = 18, the run time is 50 years
  For n = 20, the run time is 200 centuries

Standard Analysis Techniques

- Constant time statements
- Analyzing Loops
- Analyzing Nested Loops
- Analyzing Sequence of Statements
- Analyzing Conditional Statements

Constant time statements

- Simplest case: O(1) time statements
- Assignment statements of simple data types
  int x = y;
- Arithmetic operations:
  x = 5 * y + 4 - z;
- Array referencing:
  A[j] = 5;
- Array assignment:
  ∀ j, A[j] = 5;
- Most conditional tests:
  if (x < 12) ...

Analyzing Loops

- Any loop has two parts:
  - How many iterations are performed?
  - How many steps per iteration?

```
int sum = 0,j;
for (j=0; j < N; j++)
  sum = sum +j;
```

  - Loop executes N times (0..N-1)
  - 4 = O(1) steps per iteration
- Total time is N * O(1) = O(N*1) = O(N)

Analyzing Loops

- What about this **for** loop?

```
int sum =0, j;
for (j=0; j < 100; j++)
  sum = sum +j;
```

- Loop executes 100 times
- 4 = O(1) steps per iteration
- Total time is 100 * O(1) = O(100 * 1) = O(100) = O(1)

Analyzing Nested Loops

- Treat just like a single loop and evaluate each level of nesting as needed:

```
int j,k;
for (j=0; j<N; j++)
  for (k=N; k>0; k--)
    sum += k+j;
```

- Start with outer loop:
  - How many iterations? N
  - How much time per iteration? Need to evaluate inner loop
- Inner loop uses O(N) time
- Total time is N * O(N) = O(N*N) = $O(N^2)$

Analyzing Nested Loops

- What if the number of iterations of one loop depends on the counter of the other?

```
int j,k;
for (j=0; j < N; j++)
  for (k=0; k < j; k++)
    sum += k+j;
```

- Analyze inner and outer loop together:
- Number of iterations of the inner loop is:
- $0 + 1 + 2 + \ldots + (N-1) = O(N^2)$

Analyzing Sequence of Statements

- For a sequence of statements, compute their complexity functions individually and add them up

```
for (j=0; j < N; j++)
  for (k =0; k < j; k++)
    sum = sum + j*k;
for (l=0; l < N; l++)
  sum = sum -l;
cout<<“Sum=”<<sum;
```

$O(N^2)$

$O(N)$

$O(1)$

Total cost is $O(N^2) + O(N) + O(1) = O(N^2)$

SUM RULE

Analyzing Conditional Statements

What about conditional statements such as

```
if (condition)
  statement1;
else
  statement2;
```

where statement1 runs in O(N) time and statement2 runs in $O(N^2)$ time?

We use "worst case" complexity: among all inputs of size N, that is the maximum running time?

The analysis for the example above is $O(N^2)$

Time complexity familiar tasks

Task	Growth rate
Getting a specific element from a list (array)	$O(1)$
Dividing a list in half, dividing one halve in half, etc	$O(\log_2 N)$
Binary Search	$O(\log_2 N)$
Scanning (brute force search) a list	$O(N)$
Nested **`for`** loops (k levels)	$O(N^k)$
MergeSort	$O(N \log_2 N)$
BubbleSort	$O(N^2)$
Generate all subsets of a set of data	$O(2^N)$
Generate all permutations of a set of data	$O(N!)$

Thank You☺